

BPSK_mapper

```
1  import numpy as np
2
3
4  def int_to_bits_lsb(value: int, length: int) -> np.ndarray:
5
6      if value < 0:
7          raise ValueError("value >= 0 must be")
8      if value >= (1 << length):
9          raise ValueError("value size not combinian into import length")
10
11      return np.array([(value >> i) & 1 for i in range(length)],
12                      dtype=np.uint8)
13
14
15  def bits_to_int_lsb(bits):
16      value = 0
17
18      for i, bit in enumerate(bits):
19          value |= int(bit) << i
20
21      return value
22
23
24
25  class BPSK_mapper:
26
27      def __init__(self, bit_rate, E_symb_avg=1):
28
29          self._bit_rate          = bit_rate
30          self._num_bits_in_qam    = 1
31          self._symbol_rate        = self._bit_rate/self._num_bits_in_qam
32          self._T_symb              = 1/self._symbol_rate
33          self._E_symb_avg          = E_symb_avg
34          self._E_bit_avg           = E_symb_avg
35          self._mod_signals         = self._get_mod_arr()
36
37
38
39      def _get_mod_arr(self):
40          arr_signals = np.zeros(2, dtype=complex)
41
42          for numb in range(0, 2):
43              bits = int_to_bits_lsb(numb, 1)
44              arr_signals[numb] = (1/np.sqrt(2))*((1 - 2*bits[0]) + 1j*(1 - 2*bits[0]))
45
46          return arr_signals * np.sqrt(self._E_symb_avg)
47
48
```

```

49     def get_average_energy(self):
50
51         return sum(x * x for x in self._mod_signals) / 2
52
53
54     def get_baseband_signal(self, bit_sequence):
55
56         if ((np.size(bit_sequence) % self._num_bits_in_qam) != 0): raise ValueError("the number of bits is not a multiple of the number of bits per symbol")
57
58         bit_seq_sz = np.size(bit_sequence)
59
60         def baseband_signal_t(t):
61
62             n = t // self._T_symb
63             start_bits = int(n*self._num_bits_in_qam)
64             end_bits = int((n+1)*self._num_bits_in_qam)
65
66             if start_bits >= bit_seq_sz or end_bits > bit_seq_sz: raise ValueError("There aren't bits for do modulating")
67
68             cur_num = bits_to_int_lsb(bit_sequence[start_bits:end_bits])
69
70             return self._mod_signals[cur_num]
71
72         def baseband_signal(t_arr):
73             if np.isscalar(t_arr):
74                 return baseband_signal_t(t_arr)
75
76             result = np.zeros_like(t_arr, dtype=complex)
77
78             for i in range(0, np.size(t_arr)):
79                 result[i] = baseband_signal_t(t_arr[i])
80
81             return result
82
83         return baseband_signal
84
85
86     def get_modulate_sequence_and_time(self, t_start, t_end, numb_of_points,
bit_sequence):
87
88         if (t_end - t_start) <= 0: raise ValueError("t_end <= t_start")
89         if ((t_end - t_start) // self._T_symb + 1) * self._num_bits_in_qam >
np.size(bit_sequence): raise ValueError("Insufficient bits to form the baseband signal")
90
91         if numb_of_points <= 0: raise ValueError("numb_points must be > 0")
92
93         t = np.linspace(t_start, t_end, numb_of_points)
94         result = np.zeros(numb_of_points, dtype=complex)
95
96         for i in range(0, numb_of_points):

```

```

96         n = (t[i] - t[0]) // self._T_symb
97
98         if n >= np.size(numb_of_points):
99             result[i] = 0
100             continue
101
102         start_bits = int(n*self._num_bits_in_qam)
103         end_bits = int((n+1)*self._num_bits_in_qam)
104
105         cur_signal_val = bits_to_int_lsb(bit_sequence[start_bits:end_bit
s])
106
107         result[i] = self._mod_signals[cur_signal_val]
108
109     return result, t
110
111
112
113
114
115
116

```